

1. Определение проблемы

В современную эпоху стриминговых сервисов и онлайн-баз (Kinopoisk, IMDb, YouTube) пользователю доступно огромное количество фильмов и сопутствующих материалов (аннотации, постеры, обзоры, мерч). При таком разнообразии подбор «интересного» фильма занимает слишком много времени: необходимо переходить между разными платформами, искать качественные видеоразборы и вручную хранить ссылки и заметки. В итоге многие фильмы, которые пользователь хотел посмотреть, теряются, а поиск глубоких аналитических обзоров (не просто трейлеров) превращается в длительный и неудобный процесс.

Описание болей пользователей

- **Отсутствие единого списка «Смотреть позже».**
Пользователи сохраняют ссылки в закладках браузера или заметках, что неудобно: ссылки могут потеряться, надо вводить поисковую строку заново, нет возможности быстро увидеть всё, что накоплено за длительный период.
- **Трудоёмкий поиск качественных видеобзоров (разборов) на YouTube.**
На YouTube огромное количество роликов-разборов, но не все они содержательны: приходится просматривать десятки видео, разбираться в авторитетности канала и искать свежие, экспертные разборы.
- **Невозможность централизованно хранить собственные рейтинги и впечатления.**
Если зритель поставил фильму оценку (например, 8/10), он не может сохранять это внутри единого сервиса — приходится полагаться на память или сторонние приложения, что затрудняет сравнение и анализ исторических предпочтений.

Чёткая формулировка проблемы

Пользователи теряют время на фрагментарный поиск информации о фильмах и качественных обзорах на разных платформах, не имеют удобного централизованного списка «Смотреть позже» и не могут сохранить собственные оценки просмотренных картин в одном месте. Это приводит к тому, что интересные фильмы забываются, а поиск содержательных видеоразборов превращается в длительный и неудобный процесс.

2. Выработка требований

Пользовательские истории

1. Добавление фильма в «Смотреть позже»

- Как пользователь, **я хочу** находить интересный фильм (по названию или жанру) и одним кликом добавлять его в список «Смотреть позже», чтобы не забыть о фильме и иметь возможность вернуться к нему в любой момент.

2. Просмотр рекомендаций на основе рейтингов

- Как пользователь, **я хочу** выставять рейтинги (оценки) просмотренным фильмам (например, от 1 до 10), чтобы в будущем система могла предлагать похожие или востребованные фильмы, а я мог видеть список своих лучших просмотров.

3. Поиск и просмотр авторитетных видеообзоров (разборов)

- Как пользователь, **я хочу** получать ссылки на качественные YouTube-разборы выбранного фильма (анализ сюжета, режиссёрских приёмов, скрытых деталей), Чтобы не тратить время на поиск и просмотр десятков нерелевантных роликов, а сразу перейти к глубокому разбору.

4. Управление личным списком «Смотреть позже»

- Как пользователь, **я хочу** удалять фильмы из списка «Смотреть позже» и видеть временную метку добавления, Чтобы держать список актуальным и отслеживать, когда именно я запланировал просмотр.

5. Регистрация и авторизация пользователя

- Как **новый** пользователь, **я хочу** зарегистрироваться (создать учётную запись) и войти в систему по электронному адресу и паролю, Чтобы мои списки «Смотреть позже», оценки и история просмотров были доступны только мне и сохранялись между сеансами.

Оценка числа пользователей и нагрузка

Для проектирования системы принимаем следующие ориентиры:

- **Число активных пользователей:**
 - 10 000 пользователей в сутки (ориентировочно).
Это позволит выполнить расчёт нагрузки на базу данных, API-сервер и оценить требования к хранилищу.
- **Пиковая нагрузка:**
 - В пиковые часы система должна выдерживать до 100 запросов в секунду (например, вечером, когда многие возвращаются с работы и приступают к просмотру/поиску фильмов).
- **Нагрузочное соотношение R/W:**

- Около 80 % запросов приходятся на чтение (поиск фильмов, получение деталей, списка «Смотреть позже»).
- Около 20 % запросов — на запись (регистрация, добавление/удаление из списка, выставление оценки).

Период хранения и объёмы данных

1. Период хранения пользовательских данных:

- Не менее 5 лет.

Это обеспечивает хранение длительной истории просмотров, рейтингов и планов на просмотр (например, аспиранту или киноману важно иметь доступ к данным о фильмах, добавленных ещё несколько лет назад).

2. Оценка объёма хранения:

- Пользовательские данные:
 - Предполагаем, что у каждого из 10 000 активных пользователей в среднем хранится до 100 фильмов в «Смотреть позже» (включая удалённые и пересмотренные).
 - Таблица «watchlist» будет содержать порядка 1 000 000 записей (100 фильмов × 10 000 пользователей).
- Таблица оценок (ratings):
 - Допустим, каждый пользователь выставляет оценки примерно 50 фильмам (это может изменяться, но для расчётов примем 50).
 - Получаем 500 000 записей в таблице рейтингов.
- Таблица фильмов (movies):
 - Предположим, что в базе локального кэша хранится не более 100 000 уникальных фильмов (из тех, что искали пользователи).
 - Поля: `movie\_id`, `title`, `year`, `poster\_url`, `description`, `last\_sync` и т. п. Средний размер строки — ~1 КБ.
- Итоговый объём (приблизительно):
 1. Пользователи: ~10 000 записей × 200 Б на запись ≈ 2 МБ.
 2. Watchlist: 1 000 000 записей × 100 Б ≈ 100 МБ.
 3. Ratings: 500 000 записей × 100 Б ≈ 50 МБ.
 4. Movies: 100 000 записей × 1 КБ ≈ 100 МБ.
 5. Дополнительные индексы и служебные данные: ~50 МБ.

Итого (включая запас): ~300 МБ на хранение основных таблиц, что легко укладывается в возможности PostgreSQL.

3. Разработка архитектуры и детальное проектирование

3. Разработка архитектуры и детальное проектирование.....	1
3.1. Характер нагрузки на сервис.....	2
3.1.1. Соотношение R/W нагрузки.....	2
3.1.2. Объемы трафика.....	2
3.1.3. Объемы дисковой системы.....	2
3.2. Диаграммы C4.....	3
3.2.1. Контекстная диаграмма (C4 – Уровень 1).....	3
3.2.2. Контейнерная диаграмма (C4 – Уровень 2).....	4
3.3. Контракты API.....	5
3.3.1. Основные эндпоинты.....	5
3.4. Ожидаемые нефункциональные требования.....	7
3.5. Схема базы данных.....	8
3.5.1. Описание таблиц.....	8
3.5.2. Обоснование устойчивости схемы БД.....	8
3.6. Масштабирование сервиса при росте нагрузки в 10 раз.....	9
3.6.1. Горизонтальное масштабирование Backend.....	9
3.6.2. Переход к кластеру PostgreSQL.....	9

3.1. Характер нагрузки на сервис

3.1.1. Соотношение R/W нагрузки

- **Общее число пользователей (ориентировочно):**
 - 10 000 активных пользователей в сутки.
- **Число запросов в сутки (ориентировочно):**
 - ~150 000 API-запросов.
- **Средняя нагрузка (gps):**
 - $150\,000 / (24 \times 60 \times 60) \approx 1.7$ запроса в секунду.
- **Пиковая нагрузка (gps):**
 - До 5 запросов в секунду (вечерние часы).
- **Соотношение операций чтения и записи:**
 - Примерно 80 % запросов — чтение (поиск фильмов, получение деталей, списков «Смотреть позже», рейтингов).
 - Примерно 20 % запросов — запись (регистрация/логин, добавление/удаление из списка, выставление рейтинга).

3.1.2. Объемы трафика

- **Средний размер ответа на один API-запрос:** ~1 КБ (JSON с метаданными).
- **Итоговый трафик в сутки:**
 - $150\,000 \text{ запросов} \times 1 \text{ КБ} \approx 150 \text{ МБ}$.
 - С учётом повторных запросов и статических ресурсов (React-bundle, картинки) – до ~300–400 МБ/сутки.
- **Итоговый трафик в месяц:**
 - ~9–12 ГБ.

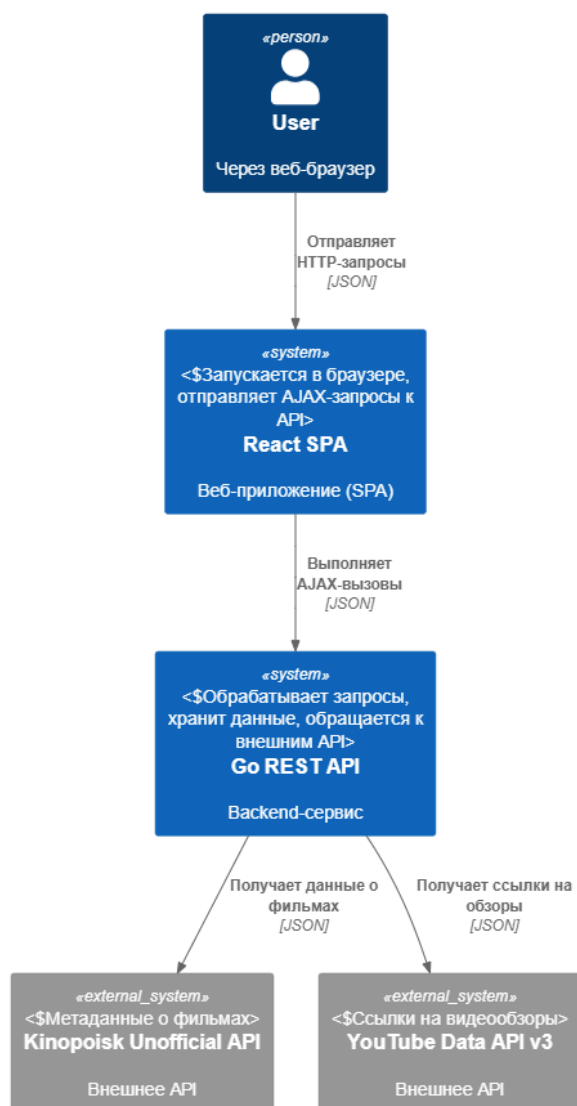
3.1.3. Объемы дисковой системы

Таблица	Количество записей	Средний размер записи	Итоговый объём
users	10 000	~200 Б	~2 МБ
watchlist	1 000 000	~100 Б	~100 МБ
ratings	500 000	~100 Б	~50 МБ
movies	100 000	~1 КБ	~100 МБ

Индексы и служебные данные	—	—	~50 МБ
ИТОГО	—	—	302 МБ ≈ 350 МБ

3.2. Диаграммы C4

3.2.1. Контекстная диаграмма (C4 – Уровень 1)

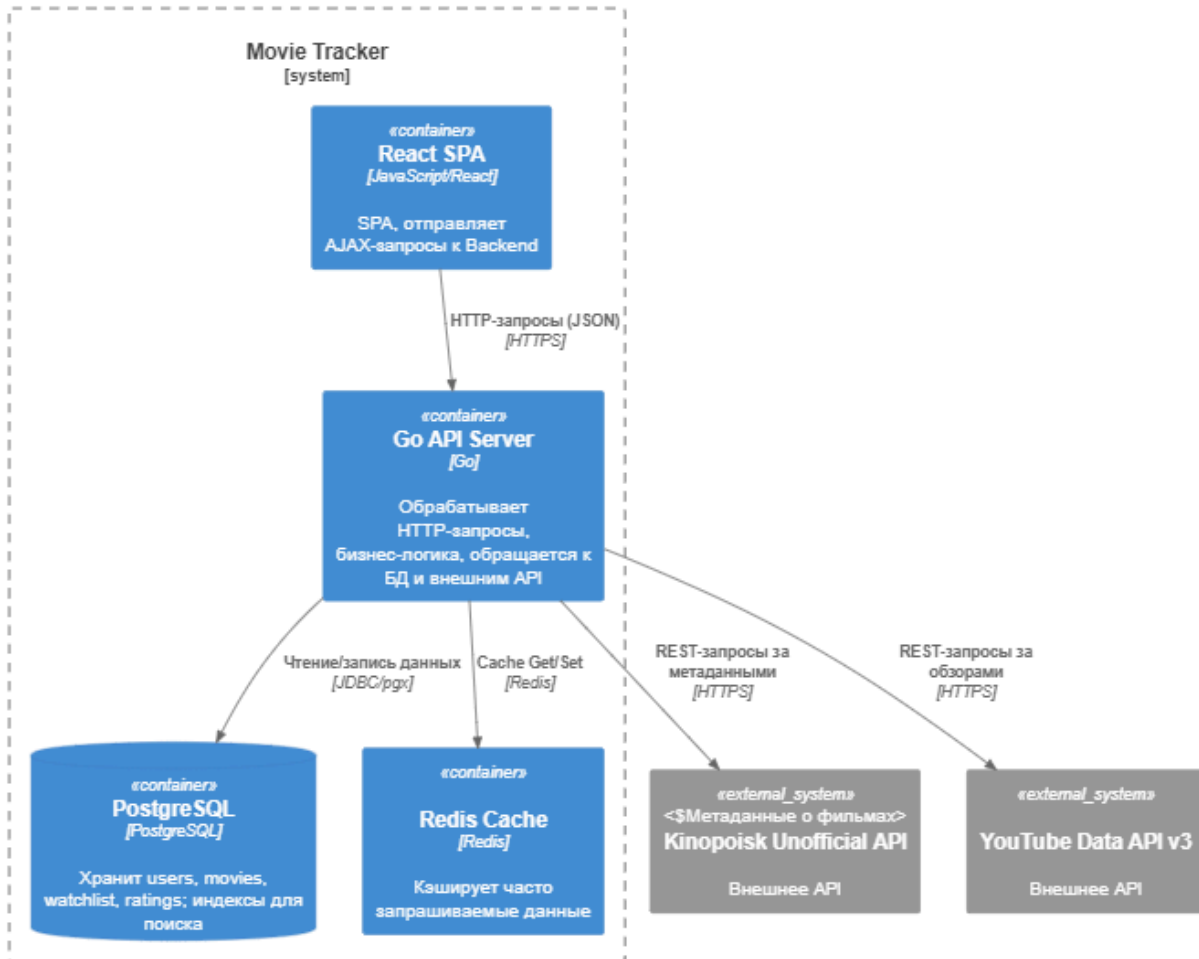


- **Компоненты:**

1. **Пользователь (User)** – через веб-браузер.
2. **Веб-приложение (React SPA)** – запускается в браузере, обращается к Backend.
3. **Backend (Go REST API)** – обрабатывает запросы от Frontend, запрашивает Kinopoisk API и YouTube API.

4. **Kinopoisk Unofficial API** – внешний сервис для получения метаданных о фильмах.
5. **YouTube Data API v3** – внешний сервис для поиска видеообзоров.

3.2.2. Контейнерная диаграмма (C4 – Уровень 2)



• Контейнеры:

1. **React SPA (Frontend Container)**
 - Развёртывается как статический набор HTML/CSS/JS (в Docker Nginx или любой статический сервер).
 - Отправляет AJAX-запросы к API Server.
2. **Go API Server (Backend Container)**
 - Обрабатывает HTTP-запросы, реализует бизнес-логику.
 - Взаимодействует с PostgreSQL.
 - Вызывает внешний Kinopoisk API и YouTube API.
 - Выполняет аутентификацию/авторизацию (JWT).
3. **PostgreSQL (Database Container)**
 - Хранит таблицы: users, movies, watchlist, ratings.
 - Индексы для ускорения поиска (по movie_id, user_id, title).
4. **(Опционально) Redis/Cache (Cache Container)**

3.3. Контракты API

3.3.1. Основные эндпоинты

1. Поиск фильмов

- **Метод:** `GET /api/movies/search?q={query}`
- **Параметры запроса:**
 - `q` (string) — поисковая строка (название, часть названия, ключевые слова).
- **Описание:**
 - Backend передаёт запрос к Kinopoisk API.
 - Возвращает JSON-массив объектов `MoviePreview` (поля: `movie_id`, `title`, `year`, `poster_url`).

2. Получение деталей фильма

- **Метод:** `GET /api/movies/{movie_id}`
- **Параметры пути:**
 - `movie_id` (integer) — идентификатор фильма в Kinopoisk.
- **Описание:**
 - Backend проверяет локальный кэш (таблица `movies`).
 - Если данных нет или устарели (старше 30 суток), выполняет запрос к Kinopoisk API, сохраняет результат.
 - Возвращает полную информацию о фильме: `title`, `year`, `genres`, `description`, `poster_url`, `rating_kp`, `rating_imdb` и др.

3. Получение видеообзора (разбора)

- **Метод:** `GET /api/movies/{movie_id}/reviews`
- **Параметры пути:**
 - `movie_id` (integer) — идентификатор фильма.
- **Описание:**
 - Backend формирует поисковый запрос «`{title}` review analysis» (или «`{title}` разбор») к YouTube API.
 - Возвращает ссылку(и) на 1–3 наиболее релевантных видео: `video_id`, `title`, `channel_title`, `thumbnail_url`.

4. Получение списка «Смотреть позже»

- **Метод:** `GET /api/users/{user_id}/watchlist`
- **Параметры пути:**
 - `user_id` (integer) — идентификатор пользователя (берётся из JWT).
- **Требуется авторизации:** Да (JWT).
- **Описание:**
 - Возвращает JSON-массив объектов `WatchlistItem`: `movie_id`, `added_at`, и краткие поля фильма (`title`, `poster_url`).

5. Добавление фильма в «Смотреть позже»

- **Метод:** `POST /api/users/{user_id}/watchlist`
- **Параметры пути:**
 - `user_id` (integer).
- **Тело (JSON):**


```
{
  "movie_id": 12345
}
```

- **Описание:**

- Проверяет, что фильм существует в таблице `movies` (если нет – запрашивает у Kinopoisk и сохраняет).
- Вставляет запись в таблицу `watchlist` (`user_id`, `movie_id`, `added_at = NOW()`).
- Возвращает статус `201 Created` и `WatchlistItem`.

6. Удаление фильма из «Смотреть позже»

- **Метод:** `DELETE /api/users/{user_id}/watchlist/{movie_id}`

- **Описание:**

- Удаляет запись из `watchlist` для данного пользователя и фильма.
- Возвращает `204 No Content`.

7. Получение списка рейтингов пользователя

- **Метод:** `GET /api/users/{user_id}/ratings`

- **Описание:**

- Возвращает JSON-массив объектов `RatingItem`: `movie_id`, `rating`, `rated_at`.

8. Выставление рейтинга фильму

- **Метод:** `POST /api/users/{user_id}/ratings`

- **Тело (JSON):**

```
{
  "movie_id": 12345,
  "rating": 8
}
```

- **Описание:**

- Проверяет существование фильма (как и раньше: из кеша или Kinopoisk).
- Если у пользователя уже была оценка для этого фильма, обновляет поле `rating` и `rated_at = NOW()`.
- Иначе вставляет новую запись.
- Возвращает `200 OK` и JSON с обновлённой информацией.

9. Регистрация пользователя

- **Метод:** `POST /api/auth/register`

- **Тело (JSON):**

```
{
  "email": "user@example.com",
  "password": "СтрогийПароль123"
}
```

- **Описание:**

- Валидирует, что email ещё не существует.
- Хэширует пароль (`bcrypt`).

- Сохраняет запись в таблице `users` (`email`, `password_hash`, `created_at = NOW()`).
- Возвращает `201 Created` и `{ "user_id": 42 }`.

10. Аутентификация (логин)

- **Метод:** `POST /api/auth/login`
- **Тело (JSON):**

```
{  
  "email": "user@example.com",  
  "password": "СтрогийПароль123"  
}
```

- **Описание:**
 - Проверяет соответствие email и пароля (сравнивает хэш).
 - При успехе генерирует JWT с коротким сроком (~1 час).
 - Возвращает `200 OK` и `{ "access_token": "...", "expires_in": 3600 }`.

3.4. Ожидаемые нефункциональные требования

1. Время отклика (Performance):

- **Чтение из собственной БД (PostgreSQL):**
 - Среднее время — до 200 мс (95 % запросов).
- **Чтение с обращением к внешним API (Kinopoisk/YouTube):**
 - Допустимое время — до 1 сек.
- **Операции записи (добавление в список, рейтинг):**
 - Среднее время — до 200–300 мс (включая транзакцию в PostgreSQL).

2. Доступность (Availability):

- Аптайм не ниже **99.9 %**.
- **Docker-контейнеры** (Backend, Frontend, БД) развёрнуты с авто-перезапуском (Restart policy) и мониторингом (healthchecks).

3. Надёжность данных (Reliability):

- **ACID-транзакции** PostgreSQL гарантируют консистентность при сбоях.
- **Резервные копии (Backups)** БД выполняются еженедельно.

4. Безопасность (Security):

- **HTTPS** на всех API-запросах.
- **JWT** для аутентификации (краткий срок жизни токена).
- **Хранение паролей** строго через bcrypt (или Argon2).
- **Ограничение доступа:** к приватным эндпоинтам (`/users/*/watchlist`, `/users/*/ratings`) только после успешного логина.
- **Валидация всех входных данных.**

5. Масштабируемость (Scalability):

- **Горизонтальное масштабирование** Backend Server (несколько контейнеров за балансировщиком).
- **Репликация БД:** одна мастер-нода (для записи) и несколько реплик (для чтения).
- **Кеширование (по возможности):**
 - Redis или встроенный кеш Go (при дальнейших улучшениях).

– Кеширование часто запрашиваемых фильмов (для снижения частоты вызовов к Kinopoisk).

3.5. Схема базы данных

3.5.1. Описание таблиц

1. users

- `user_id` (SERIAL PK) — уникальный идентификатор пользователя.
- `email` (VARCHAR, UNIQUE, NOT NULL) — адрес электронной почты.
- `password_hash` (VARCHAR, NOT NULL) — bcrypt-хэш пароля.
- `created_at` (TIMESTAMP, DEFAULT NOW()) — дата регистрации.

2. movies

- `movie_id` (INTEGER PK) — идентификатор фильма из Kinopoisk.
- `title` (VARCHAR, NOT NULL) — название.
- `year` (INTEGER) — год выпуска.
- `poster_url` (VARCHAR) — ссылка на постер.
- `description` (TEXT) — краткое описание сюжета.
- `last_sync` (TIMESTAMP) — дата последнего обновления из Kinopoisk.

3. watchlist

- `watchlist_id` (SERIAL PK) *или* составной ключ (`user_id`, `movie_id`) (см. коммент ниже).
- `user_id` (INTEGER, FK → `users.user_id`).
- `movie_id` (INTEGER, FK → `movies.movie_id`).
- `added_at` (TIMESTAMP, DEFAULT NOW()).

4. ratings

- `rating_id` (SERIAL PK) *или* составной ключ (`user_id`, `movie_id`).
- `user_id` (INTEGER, FK → `users.user_id`).
- `movie_id` (INTEGER, FK → `movies.movie_id`).
- `rating` (SMALLINT, NOT NULL, CHECK (rating BETWEEN 1 AND 10)).
- `rated_at` (TIMESTAMP, DEFAULT NOW()).

3.5.2. Обоснование устойчивости схемы БД

- **ACID-транзакции (PostgreSQL):**

– При добавлении/удалении записей (watchlist, ratings) операции выполняются в транзакциях. При сбое транзакция откатывается.

– СУБД автоматически поддерживает целостность ссылок (FOREIGN KEY с ON DELETE CASCADE).

- **Индексы:**

– Индекс на `users.email` для быстрого поиска при логине (UNIQUE).

– Индексы на `watchlist (user_id)` и `(movie_id)` для быстрого получения

списка.

– Полнотекстовый индекс на `movies.title` (GIN-индекс) для ускорения поиска.

- **Локальный кэш фильмов:**

– Данные фильмов из Kinopoisk сохраняются в таблицу `movies` (с полем `last_sync`).

– При запросе `/movies/{id}` сначала проверяем `last_sync`; если давно не обновлялось (> 30 дней), запрашиваем свежие данные из Kinopoisk и обновляем запись.

– Это снижает количество вызовов к внешнему API и сохраняет QoS (когда Kinopoisk временно недоступен, данные из локального кеша всё ещё есть).

- **Резервное копирование и репликация:**

– Регулярные бэкапы (например, `pg_dump` «ночью»).

– Возможность настроить репликацию (мастер $\rightarrow$ реплика) для повышения отказоустойчивости.

3.6. Масштабирование сервиса при росте нагрузки в 10 раз

3.6.1. Горизонтальное масштабирование Backend

Docker-контейнеризация:

– Каждый компонент (React SPA, Go API Server, PostgreSQL) упакован в собственный Docker-контейнер.

- – При росте нагрузки в 10 раз увеличиваем число экземпляров сервиса `api` и `frontend` (через `docker-compose scale api=3 frontend=3`).

- **Балансировщик нагрузки (Load Balancer):**

– Для распределения запросов между несколькими инстансами Backend API ставится Nginx или Traefik (в Docker).

3.6.2. Переход к кластеру PostgreSQL

- **Основная нода (Master) + реплики (Slaves)**

– На этапе разработки достаточно один контейнер PostgreSQL.

– При росте нагрузки на чтение ($10\times$) организуем репликацию:

1. Основная нода принимает записи (Insert/Update).
2. Реплики обслуживают только SELECT-запросы (чтение).
3. Balance между репликами можно осуществлять через PgBouncer.